COMPUTER ARCHITECTURE

CPU Performance Analysis

Understanding Execution Time, CPI, MIPS & Performance Metrics

Performance Metrics

Numerical Solutions

Real-World Examples

Contents

01 **Introduction to CPU Performance**
Understanding performance fundamentals and key factors

02 **CPU Performance Equation**
The fundamental formula and its components

03 **Cycles Per Instruction (CPI)**
Understanding instruction efficiency metrics

04 **CPU Execution Time**
Step-by-step numerical calculations

05 **MIPS Metric**
Million Instructions Per Second explained

06 **Processor Comparison**
Real-world performance comparisons

07 **SPEC Benchmark**
Industry-standard performance measurement

08 **Amdahl's Law**
Understanding performance limits

CHAPTER 01

Introduction to CPU Performance

What is CPU Performance?

CPU Performance measures how quickly a processor can execute programs. It's determined by the total time required to complete a task, influenced by three critical factors working together.

Key Insight:

Performance is **inversely proportional** to execution time. Lower execution time = Better performance.

Why Performance Matters

- **User Experience:** Faster response times improve productivity
- **Cost Efficiency:** Better performance per watt saves energy
- **Scalability:** Handle more workloads with fewer resources
- **Competitive Edge:** Critical in data centers and HPC

Three Key Factors

1 Instruction Count (IC)

Total number of machine instructions executed by the program. Determined by program algorithm, compiler efficiency, and ISA.

2 CPI (Cycles Per Instruction)

Average clock cycles needed per instruction. Varies by instruction type and CPU architecture (pipelining, cache, etc.).

3 Clock Rate / Cycle Time

Determines how fast each cycle completes. Higher clock rate = shorter cycle time. Measured in GHz (billions of cycles/sec).

CHAPTER 02

The CPU Performance Equation

Fundamental Performance Equation

$$\text{CPU Time} = IC \times CPI \times \text{Clock Cycle Time}$$

or equivalently: $\text{CPU Time} = \frac{(IC \times CPI)}{\text{Clock Rate}}$

This equation is the foundation of CPU performance analysis, showing how three factors combine to determine execution time.

IC Instruction Count

Total instructions executed. Affected by:

- Algorithm efficiency
- Compiler optimization
- Instruction Set Architecture

Example: A loop running 1000 times with 5 instructions = 5000 IC

CPI Cycles Per Instruction

Average cycles per instruction. Affected by:

- CPU microarchitecture
- Memory hierarchy
- Pipeline efficiency

Typical Values: RISC: 1-2, CISC: 2-4, Complex ops: 10+

T Clock Cycle Time

Duration of one clock cycle. Related to clock rate:

$$T = 1 / f$$

Cycle Time = 1 / Clock Rate

Example: 3 GHz CPU → T = 0.333 ns

CHAPTER 03

Understanding CPI (Cycles Per Instruction)

What is CPI?

CPI (Cycles Per Instruction) measures the average number of clock cycles required to execute one instruction. It reflects CPU efficiency— **lower CPI means better performance** .

Key Formula:

$$CPI = \frac{\text{Total Clock Cycles}}{\text{Instruction Count}}$$

CPI Varies by Instruction Type

Different instructions require different cycles based on complexity:

Instruction Type	Typical CPI	Description
Integer Add/Sub	1	Simple ALU operation
Integer Multiply	3-4	More complex arithmetic
Load/Store	2-6	Memory access operations
Floating Point	4-20	Complex FP operations
Branch/Jump	1-3	Control flow changes

Weighted Average CPI

For programs with mixed instructions:

$$CPI_{avg} = \Sigma(CPI_i \times F_i)$$

where F_i = fraction of instruction type i

Example Calculation:

Program: 60% ALU (CPI=1), 30% Load (CPI=4), 10% Branch (CPI=2)

CPI = (0.6×1) + (0.3×4) + (0.1×2) = 2.0

Factors Affecting CPI

- **Pipeline Depth:** Deeper pipelines can reduce CPI but increase branch penalties
- **Cache Performance:** Cache misses increase CPI significantly
- **Branch Prediction:** Mispredictions waste cycles
- **Out-of-Order Execution:** Reduces effective CPI by parallelizing

CPU Execution Time - Numerical Examples

1 Basic Execution Time

Problem:

A program has 1 billion instructions, CPI = 2.5, and clock cycle time = 1 ns. Calculate CPU execution time.

Solution:

CPU Time = IC × CPI × Cycle Time

$= 10^9 \times 2.5 \times 1$ ns

$= 2.5 \times 10^9$ ns

= 2.5 seconds

Answer: 2.5 seconds

2 With Clock Rate

Problem:

IC = 2×10^9, CPI = 3, Clock Rate = 3 GHz. Find: (a) Clock cycle time, (b) CPU execution time.

Solution:

(a) Cycle Time = 1 / f

$= 1 / (3 \times 10^9)$

= 0.333 ns

(b) CPU Time = (IC × CPI) / f

$= (2\times10^9 \times 3) / (3\times10^9)$

= 2 seconds

Answer: 0.333 ns, 2 seconds

3 Multi-Instruction Type Program

Problem:

A program has three instruction classes with different CPI values:

· Arithmetic (CPI=1.5):	500 million
· Branch (CPI=2.0):	200 million
· Memory (CPI=2.5):	300 million

Solution:

$T_arith = (500\times10^6 \times 1.5) / (3\times10^9)$

= 0.25 sec

$T_branch = (200\times10^6 \times 2.0) / (3\times10^9)$

= 0.133 sec

$T_memory = (300\times10^6 \times 2.5) / (3\times10^9)$

= 0.25 sec

Total Time = 0.25 + 0.133 + 0.25

Answer: 0.633 seconds

CHAPTER 05

MIPS - Million Instructions Per Second

What is MIPS?

MIPS (Million Instructions Per Second) measures how many millions of instructions a CPU can execute in one second. It's a throughput metric that indicates raw processing speed.

Key Formulas:

$$\text{MIPS} = \frac{\text{Instruction Count}}{\text{Execution Time} \times 10^6}$$

$$\text{MIPS} = \frac{\text{Clock Rate}}{\text{CPI} \times 10^6}$$

Limitations of MIPS

- **Ignores instruction complexity:** Not all instructions do equal work
- **ISA dependent:** Different architectures have different instruction costs
- **Can be misleading:** Higher MIPS doesn't always mean better performance
- **Called "Meaningless Indicator of Performance"** by critics

Numerical Example

Problem:

Calculate MIPS for: IC = 3×10^8, CPU Time = 2 seconds

Solution:

MIPS = IC / (CPU Time × 10^6)

= $(3 \times 10^8) / (2 \times 10^6)$

= 300 / 2

= 150 MIPS

Answer: 150 MIPS

Real-World Insight

Classic Example:

A compiler optimization reduces instruction count by eliminating simple instructions (low CPI). Result:

- Program runs **faster**
- MIPS rating **decreases**
- Shows MIPS can be misleading!

Better Alternative:

Use **execution time** or **SPEC benchmarks** for accurate comparisons.

Processor Comparison - Real Examples

Comparing Two Processors

Processor A

CPI:	2.0
Clock Rate:	2.5 GHz

Processor B

CPI:	1.5
Clock Rate:	2.0 GHz

Question: Which processor has better performance for the same instruction count?

Assume IC = 10^9 instructions for both processors.

Solution for Processor A

CPU Time = (IC × CPI) / Clock Rate

$= (10^9 \times 2.0) / (2.5 \times 10^9)$

= 2.0 / 2.5

= 0.8 seconds

Execution Time: 0.8 sec

Solution for Processor B

CPU Time = (IC × CPI) / Clock Rate

$= (10^9 \times 1.5) / (2.0 \times 10^9)$

= 1.5 / 2.0

= 0.75 seconds

Execution Time: 0.75 sec

Result: Processor B Wins!

Despite having a **lower clock rate** (2.0 vs 2.5 GHz), Processor B is **6.25% faster** due to better CPI (1.5 vs 2.0).

Speedup

1.067×

SPEC Benchmark - Industry Standard

What is SPEC?

SPEC (Standard Performance Evaluation Corporation) is an industry consortium that develops standardized benchmarks for measuring computer system performance.

Why SPEC is Better:

✓ Uses real application code
✓ Tests integer & floating-point
✓ Normalized to reference system
✓ Industry-accepted standard

CHAPTER 08

Amdahl's Law - Performance Limits

What is Amdahl's Law?

Proposed by Gene Amdahl in 1967, this law calculates the theoretical speedup when improving part of a system. It shows that **performance improvement is limited by the portion that cannot be improved** .

Key Insight:

Even with infinite resources, the sequential portion becomes a bottleneck, setting a hard limit on speedup.

The Formula

$$\text{Speedup} = \frac{1}{(1 - f) + f/s}$$

Where f = fraction enhanced, s = speedup factor

(1 - f): Sequential portion (unchanged)

f/s: Enhanced portion (divided by speedup)

Maximum Speedup: 1 / (1 - f) as s → ∞

Problem Statement

> A program runs in **100 seconds**. **70%** of it can be parallelized and upgraded to run **5× faster**. What is the overall speedup?

Given

f = 0.70 — 70% of program is enhanced

s = 5 — enhanced part runs 5× faster

(1 − f) = 0.30 — 30% is sequential (unchanged)

Original Time = 100 sec

Applying the Formula

Speedup = 1 / [(1 − f) + (f / s)]

= 1 / [(1 − 0.70) + (0.70 / 5)]

= 1 / [0.30 + 0.14]

= 1 / 0.44

= 2.27×

CHAPTER 09

Real-World Applications & Case Studies

Compiler Optimization

Understanding CPI helps compiler designers choose instruction sequences that minimize execution time, not just instruction count.

Example:
A compiler might choose 6 simple instructions (CPI=1) over 4 complex ones (CPI=2) because 6×1 < 4×2 cycles.

Cloud Computing Selection

Businesses use SPEC benchmarks to select cost-effective cloud instances based on actual workload performance.

Decision Factor:
Price/performance ratio using SPEC scores helps optimize cloud spending for compute-intensive applications.

Hardware Design Trade-offs

CPU architects use performance equations to balance clock rate vs. CPI when designing pipelines and execution units.

Classic Case:
Pentium 4 had high clock rate but deep pipeline (high CPI), making it slower than Pentium III in some workloads.

Gaming Performance

Game developers optimize for low CPI by minimizing cache misses and branch mispredictions in critical code paths.

Optimization:
Physics engines use data-oriented design to improve cache locality, reducing memory access cycles (lower effective CPI).

Modern CPU Architecture Optimizations

Out-of-Order Execution
Reduces effective CPI

Large Caches
Minimizes memory stalls

Branch Prediction
Reduces pipeline flushes

SIMD Instructions
Higher throughput

Key Takeaways

1 CPU Performance Equation

$$\text{CPU Time} = IC \times CPI \times T$$

All three factors (Instruction Count, CPI, Clock Cycle Time) must be considered together. Focusing on just one can be misleading.

2 CPI is Critical

Lower CPI means better performance. Modern CPUs use techniques like pipelining, out-of-order execution, and branch prediction to reduce CPI.

Remember: A processor with lower clock rate but better CPI can outperform a higher clocked competitor.

3 MIPS Has Limitations

MIPS can be misleading because it ignores instruction complexity and varies by ISA. It's been called "Meaningless Indicator of Performance Statistic."

Better: Use execution time or SPEC benchmarks for accurate comparisons.

4 Amdahl's Law

Performance improvement is limited by the sequential portion. Maximum speedup = 1 / (1 - f), where f is the parallelizable fraction.

Implication: Focus optimization efforts on the most time-consuming portions of code.

Final Thought

CPU performance analysis requires a holistic approach. Clock speed alone doesn't determine performance—CPI and instruction count are equally important. Use industry-standard benchmarks like SPEC for meaningful comparisons.

Thank You!